SQL JOINs - Visual Guide

How Tables Merge and Rows Match

Prepared for DTank54 Group Students
Oracle HR Schema - A1 English Level

What is a JOIN?

A JOIN combines rows from two or more tables based on a related column between them. In a database, data is stored in many tables. Each table has information about one thing. But in real life, we often need information from more than one table at the same time. For example, to see an employee name with their department name, we need data from the EMPLOYEES table and the DEPARTMENTS table. A JOIN helps us put these two tables together.

Think of it like this: you have a list of employee names in one paper and a list of department names in another paper. Both papers have a department ID number. A JOIN matches the department ID from the first paper with the department ID from the second paper and puts the information together on one page.

In the HR schema, the EMPLOYEES table has department_id. The DEPARTMENTS table also has department_id. This common column is the "bridge" that connects the two tables. We call this bridge a **foreign key**.

How HR Tables Connect

In the HR schema, tables are connected through shared columns. The diagram below shows the main connections. The department_id column in EMPLOYEES connects to the department_id column in DEPARTMENTS. This is the most common JOIN you will use. Other connections exist too, like manager_id in EMPLOYEES connecting to employee_id in the same table (this is called a Self JOIN).

Table A	Column	Table B	Column
EMPLOYEES	department_id	DEPARTMENTS	department_id
EMPLOYEES	job_id	JOBS	job_id
EMPLOYEES	manager_id	EMPLOYEES	employee_id
DEPARTMENTS	location_id	LOCATIONS	location_id
LOCATIONS	country_id	COUNTRIES	country_id
COUNTRIES	region_id	REGIONS	region_id

This table shows which columns connect which tables. When you write a JOIN, you use these columns in the ON clause. For example: `e.department_id = d.department_id` means "match rows where the department_id in EMPLOYEES equals the department_id in DEPARTMENTS".

Basic JOIN Syntax

Here is the general syntax for a JOIN. The SELECT part says which columns you want to see. The FROM part says the first table. The JOIN part says the second table. The ON part says how the two tables are connected (which columns to match).

```
SELECT table1.column1, table2.column2
FROM table1
JOIN table2
  ON table1.common_column = table2.common_column;
```

The keyword JOIN is the same as INNER JOIN. Oracle also supports table aliases (short names for tables). For example, instead of writing EMPLOYEES every time, you can write 'e'. So the syntax becomes: `e.department_id = d.department_id` where 'e' is EMPLOYEES and 'd' is DEPARTMENTS.

Section 1: INNER JOIN

Idea: Only the matching rows from both tables.

INNER JOIN returns only the rows that have a match in BOTH tables. If a row in the left table has no match in the right table, it is NOT shown. If a row in the right table has no match in the left table, it is NOT shown either. Think of it like the middle part of a Venn diagram - only the overlap.

Step 1: Look at the two tables

emp_id	first_name	dept_id
101	Neena	90
102	Lex	90
103	Alexander	60
104	Bruce	60
107	Diana	NULL

dept_id	dept_name
60	IT
90	Executive
99	Unknown

Left Table (green rows = have a match)

Step 2: What the JOIN returns

first_name	dept_id	dept_name
Neena	90	Executive
Lex	90	Executive
Alexander	60	IT
Bruce	60	IT

Result Table (green = matched rows, orange = unmatched rows)

SQL Query on HR Schema:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
```

```
ORDER BY e.last_name;
```

What happens in the HR schema: This query shows the employee name and their department name. The JOIN matches department_id in EMPLOYEES with department_id in DEPARTMENTS. Only employees who have a valid department_id are shown. If an employee has NULL department_id, they will not appear in the result.

> Key point: INNER JOIN only returns rows that exist in BOTH tables. Department 99 (Unknown) does not appear because no employee works there. Diana (dept_id = NULL) does not appear because NULL cannot match anything.

Section 2: LEFT (OUTER) JOIN

Idea: All rows from the LEFT table + matching rows from the right table.

LEFT JOIN returns ALL rows from the left table, even if there is no match in the right table. If a row in the left table has no match in the right table, the result shows NULL values for the right table columns. This is useful when you want to see ALL employees, including those who do not have a department yet.

Step 1: Look at the two tables

emp_id	first_name	dept_id
101	Neena	90
103	Alexander	60
107	Diana	NULL

dept_id	dept_name
60	IT
90	Executive

Left Table (green rows = have a match)

Step 2: What the JOIN returns

first_name	dept_id	dept_name
Neena	90	Executive
Alexander	60	IT
Diana	NULL	NULL

Result Table (green = matched rows, orange = unmatched rows)

SQL Query on HR Schema:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
ORDER BY e.last_name;
```

What happens in the HR schema: This query shows ALL employees, even those who do not belong to any department. Diana has NULL department_id, so the department_name shows as NULL. But Diana still appears in the result because LEFT JOIN keeps all rows from the left table (EMPLOYEES).

Key point: LEFT JOIN keeps ALL rows from the first table. Unmatched rows get NULL for the second table columns. This is very useful for finding missing data - for example, find all employees who do not have a department: add WHERE d.department_id IS NULL at the end.

Section 3: RIGHT (OUTER) JOIN

Idea: All rows from the RIGHT table + matching rows from the left table.

RIGHT JOIN returns ALL rows from the right table, even if there is no match in the left table. If a row in the right table has no match in the left table, the result shows NULL values for the left table columns. This is useful when you want to see ALL departments, including those that have no employees yet.

Step 1: Look at the two tables

emp_id	first_name	dept_id
101	Neena	90
103	Alexander	60

dept_id	dept_name
60	IT
90	Executive
99	Unknown

Left Table (green rows = have a match)

Step 2: What the JOIN returns

first_name	dept_id	dept_name
Neena	90	Executive
Alexander	60	IT
NULL	99	Unknown

Result Table (green = matched rows, orange = unmatched rows)

SQL Query on HR Schema:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
RIGHT JOIN departments d
  ON e.department_id = d.department_id
ORDER BY d.department_name;
```

What happens in the HR schema: This query shows ALL departments, even those that have no employees. Department 99 (Unknown) appears in the result with NULL for the employee name because no employee works in that department. This helps managers see which departments are empty.

Key point: RIGHT JOIN keeps ALL rows from the second table. Unmatched rows get NULL for the first table columns. Tip: You can swap the table order and use LEFT JOIN to get the same result. Many developers prefer LEFT JOIN because it is easier to read.

Section 4: FULL OUTER JOIN

Idea: All rows from BOTH tables. Matching rows are joined, unmatched rows show NULL.

FULL OUTER JOIN returns ALL rows from both tables. If a row in the left table has no match in the right table, the right columns show NULL. If a row in the right table has no match in the left table, the left columns show NULL. This JOIN shows everything - nothing is left out.

Step 1: Look at the two tables

emp_id	first_name	dept_id
101	Neena	90
103	Alexander	60
107	Diana	NULL

dept_id	dept_name
60	IT
90	Executive
99	Unknown

Left Table (green rows = have a match)

Step 2: What the JOIN returns

first_name	dept_id	dept_name
Neena	90	Executive
Alexander	60	IT
NULL	99	Unknown
Diana	NULL	NULL

Result Table (green = matched rows, orange = unmatched rows)

SQL Query on HR Schema:

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
FULL OUTER JOIN departments d
  ON e.department_id = d.department_id
ORDER BY d.department_name;
```

What happens in the HR schema: This query shows ALL employees and ALL departments. Employees without a department show NULL for department_name. Departments without employees show NULL for employee name. This is the most complete view of the data.

> Key point: FULL OUTER JOIN shows everything from both tables. No data is left out. This is useful when you want a complete picture - for example, to find employees with no department AND departments with no employees in one query.

Section 5: CROSS JOIN (Cartesian Product)

Idea: Every row from the left table pairs with every row from the right table. No matching condition is needed.

CROSS JOIN creates every possible combination of rows from both tables. If the left table has 3 rows and the right table has 4 rows, the result has 3 x 4 = 12 rows. There is no ON clause because every row matches with every other row. This can create very large result sets, so use it carefully.

Example:

size
Small
Medium
Large

color
Red
Blue

Result (3 sizes x 2 colors = 6 rows):

size	color
Small	Red
Small	Blue
Medium	Red
Medium	Blue
Large	Red
Large	Blue

SQL Query on HR Schema:

```
SELECT e.first_name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

What happens: Every employee is paired with every department. If there are 107 employees and 11 departments, the result has 107 x 11 = 1,177 rows. This is usually not what you want in a real query, but it can be useful for generating all possible combinations.

> Key point: CROSS JOIN has no ON clause. It pairs every row with every row. Be careful - if both tables are large, the result can be millions of rows.

Section 6: Self JOIN - Detailed Explanation

6.1 What is a Self JOIN?

A Self JOIN is not a special SQL command. There is no "SELF JOIN" keyword in SQL. A Self JOIN is simply a regular JOIN where you join a table with **itself**. Because the same table appears twice in the query, you MUST give it two different aliases (short names). Without aliases, Oracle would not know which copy of the table you are referring to.

Real-life example: Think about a family tree. Every person has a name and a parent_name. If you want to find who is the parent of each person, you need to look at the same list twice - once to find the person, and once to find their parent. This is exactly what a Self JOIN does. You look at the same data from two different points of view at the same time.

In the HR schema, the EMPLOYEES table has a column called manager_id. This column stores the employee_id of the person who is the manager. So, the manager of an employee is also an employee in the same table. To

show the employee name AND the manager name together, we need to read the EMPLOYEES table twice - once as the employee (alias "e") and once as the manager (alias "m").

6.2 Why Do We Need a Self JOIN?

Look at the EMPLOYEES table below. Each row has an employee_id, a first_name, and a manager_id. The manager_id is just a number. It tells you the employee_id of the manager, but it does NOT tell you the manager name. To find the manager name, you need to look up that employee_id in the same table.

employee_id	first_name	manager_id
101	Neena	NULL
102	Lex	101
103	Alexander	102
104	Bruce	103
105	David	103

The problem: Look at row 3 (Alexander). His manager_id is 102. But who is employee 102? You need to look at the table again and find the row where employee_id = 102. That row shows Lex. So Lex is Alexander manager. But this manual lookup is slow and hard to do in SQL. A Self JOIN does this lookup automatically.

The solution: A Self JOIN lets Oracle do this lookup for you. It matches the manager_id in one copy of the table with the employee_id in another copy. The result shows both names side by side.

6.3 Step-by-Step: How Self JOIN Works

Let us see exactly what happens when Oracle runs a Self JOIN. We will follow the process step by step with color coding so you can see how each row is matched.

Step 1: Make Two Copies of the Same Table

Oracle creates two "virtual copies" of the EMPLOYEES table. The first copy is called "e" (employee view) and the second copy is called "m" (manager view). Both copies have the same data, but they play different roles. The "e" copy represents the employee, and the "m" copy represents the manager.

e.employee_id	e.first_name	e.manager_id
101	Neena	NULL
102	Lex	101
103	Alexander	102
104	Bruce	103
105	David	103

m.employee_id	m.first_name
101	Neena
102	Lex
103	Alexander
104	Bruce
105	David

Copy "e" (blue header) = Employee view. Copy "m" (purple header) = Manager view. Both are the SAME EMPLOYEES table, just with different names.

Step 2: Match Each Row

Now Oracle matches every row in "e" with every row in "m" using the condition: `e.manager_id = m.employee_id`. This means: for each employee, find the row in the manager copy where the employee_id equals the employee manager_id. Let us follow each row one by one.

Employee (e)	e.manager_id	Match in "m"	Result
Neena (e)	manager_id = NULL	NULL never equals any number, so there is no match in "m".	Neena has no manager. Result: manager name = NULL.
Lex (e)	manager_id = 101	Row 1 in "m" has employee_id = 101, first_name = Neena.	Lex manager is Neena. Result: manager name = Neena.
Alexander (e)	manager_id = 102	Row 2 in "m" has employee_id = 102, first_name = Lex.	Alexander manager is Lex. Result: manager name = Lex.
Bruce (e)	manager_id = 103	Row 3 in "m" has employee_id = 103, first_name = Alexander.	Bruce manager is Alexander. Result: manager name = Alexander.
David (e)	manager_id = 103	Row 3 in "m" has employee_id = 103, first_name = Alexander.	David manager is Alexander. Result: manager name = Alexander.

Orange row = no match (NULL manager_id). Green/white rows = matched successfully.

Step 3: Final Result

After all matching is done, Oracle creates the final result. Because we used LEFT JOIN, Neena (who has no match) still appears in the result with NULL for the manager name. Here is what the output looks like:

Employee Name	Employee ID	Manager Name	Manager ID
Neena	101	NULL	(no manager)
Lex	102	Neena	101
Alexander	103	Lex	102
Bruce	104	Alexander	103
David	105	Alexander	103

Notice that both Bruce and David have the same manager (Alexander). This is correct because two employees can share the same manager. Also notice that Neena shows NULL for the manager name because she is the top-level boss - she has no manager.

6.4 SQL Query - Employee and Manager Names

Here is the Self JOIN query. Read it line by line. We will explain each part.

```
SELECT e.first_name || ' works for ' || m.first_name
  AS "Reporting Line"
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

Line-by-Line Explanation:

Line	Code	What It Means

1	`e.first_name || ' works for ' || m.first_name`	Join the employee first name with the text " works for " and the manager first name. The "e." means take first_name from the employee copy. The "m." means take first_name from the manager copy.
2	`AS "Reporting Line"`	Give the joined text a column name. Because the name has a space, we use double quotes.
3	`FROM employees e`	Start with the EMPLOYEES table and call it "e" (employee copy).
4	`LEFT JOIN employees m`	Join the same EMPLOYEES table again, but this time call it "m" (manager copy). We use LEFT JOIN so that employees with no manager (like the CEO) are still shown.
5	`ON e.manager_id = m.employee_id`	This is the most important line! It says: match the employee manager_id with the manager employee_id. In other words, "find the manager row where the manager employee_id equals the employee manager_id".
6	`ORDER BY e.employee_id`	Sort the result by employee_id so the output is in order.

6.5 Common Mistakes with Self JOIN

Students often make these mistakes when writing a Self JOIN. Read them carefully to avoid the same errors.

Mistake 1: Forgetting aliases

```
FROM employees JOIN employees ON manager_id = employee_id
```

WRONG - Oracle does not know which table manager_id and employee_id come from. You MUST use aliases like 'e' and 'm'.

Mistake 2: Using INNER JOIN instead of LEFT JOIN

```
FROM employees e JOIN employees m ON e.manager_id = m.employee_id
```

This works but the CEO (who has no manager) will NOT appear in the result. Use LEFT JOIN to keep ALL employees, including those with NULL manager_id.

Mistake 3: Mixing up the ON condition

```
ON e.employee_id = m.employee_id
```

WRONG - This matches every employee with themselves (same ID). The correct condition is e.manager_id = m.employee_id (match the employee's manager_id with the manager's employee_id).

Mistake 4: Forgetting the table alias prefix on columns

```
SELECT first_name, first_name FROM employees e LEFT JOIN employees m ...
```

WRONG - Both copies have a first_name column. Oracle does not know which one you want. You MUST write e.first_name and m.first_name.

6.6 More Self JOIN Examples on HR Schema

Self JOINs are useful in many situations. Here are more examples using the HR schema. Each example solves a different business problem.

Example A: Find Co-Workers (Same Manager)

You want to find which employees share the same manager. This helps you understand team structure. The idea is: two employees are co-workers if they have the same manager_id.

```
SELECT e1.first_name || ' and ' || e2.first_name
  AS "Co-Workers", m.first_name AS "Their Manager" FROM employees e1
JOIN employees e2
  ON e1.manager_id = e2.manager_id
AND e1.employee_id < e2.employee_id
LEFT JOIN employees m
  ON e1.manager_id = m.employee_id
WHERE e1.manager_id IS NOT NULL
ORDER BY m.first_name;
```

How it works: We use THREE copies of EMPLOYEES. e1 and e2 are the two co-workers, and m is their manager. The condition e1.manager_id = e2.manager_id finds pairs with the same manager. The condition e1.employee_id < e2.employee_id prevents matching an employee with themselves and avoids showing duplicate pairs (like "Bruce and David" AND "David and Bruce").

Example B: Employees Who Earn More Than Their Manager

This is a classic Self JOIN problem. A company may want to find employees who earn more than their direct manager. This could be because the employee was hired recently at a higher salary, or because the manager has a different salary structure.

```
SELECT e.first_name AS "Employee", e.salary AS "Employee Salary", m.first_name AS
"Manager", m.salary AS "Manager Salary", e.salary - m.salary AS "Difference" FROM
employees e JOIN employees m
  ON e.manager_id = m.employee_id WHERE e.salary > m.salary
ORDER BY (e.salary - m.salary) DESC;
```

How it works: The Self JOIN connects each employee with their manager. Then the WHERE clause filters to show only rows where the employee salary is greater than the manager salary. The ORDER BY shows the biggest salary difference first. This is a very common interview question.

Example C: Show Manager Chain (Skip-Level)

Sometimes you need to show not just the direct manager, but also the manager manager (the skip-level manager). In a company hierarchy, your boss has a boss too. This example shows the employee, their direct manager, and the skip-level manager.

```
SELECT e.first_name AS "Employee", m.first_name AS "Direct Manager", sm.first_name AS
"Skip-Level Manager" FROM employees e LEFT JOIN employees m
  ON e.manager_id = m.employee_id LEFT JOIN employees sm
  ON m.manager_id = sm.employee_id ORDER BY e.employee_id;
```

How it works: This query uses THREE copies of EMPLOYEES. "e" is the employee, "m" is the direct manager (e.manager_id = m.employee_id), and "sm" is the skip-level manager (m.manager_id = sm.employee_id). For example, if Bruce reports to Alexander, and Alexander reports to Lex, then Bruce = "Employee", Alexander = "Direct Manager", and Lex = "Skip-Level Manager". You can add more JOINs to go even higher in the chain.

6.7 Self JOIN vs Regular JOIN - What is the Difference?

Many students ask: "How is a Self JOIN different from a regular JOIN?" The answer is simple. A Self JOIN IS a regular JOIN. The only difference is that both tables in the JOIN are the same table. Everything else is exactly the same: you still need aliases, an ON condition, and you can use INNER JOIN or LEFT JOIN. Look at the comparison below.

Feature	Regular JOIN	Self JOIN
Number of tables	Two different tables	Same table, two aliases
FROM clause	`FROM emp e JOIN dept d`	`FROM emp e JOIN emp m`
ON clause	`e.dept_id = d.dept_id`	`e.mgr_id = m.emp_id`
Need aliases?	Recommended	REQUIRED (must have 2 aliases)
Common use in HR	Employee + Department name	Employee + Manager name
Is it a SQL keyword?	JOIN is a keyword	No special keyword

6.8 Key Takeaways for Self JOIN

1. Self JOIN = Same table, two aliases. You always need two different aliases (like "e" and "m") because Oracle must distinguish between the two copies.

2. The ON condition connects different columns in the same table. For employee-manager: e.manager_id = m.employee_id. The left column is the foreign key, the right column is the primary key.

3. Use LEFT JOIN, not INNER JOIN. LEFT JOIN keeps the top-level person (like CEO) who has NULL manager_id. INNER JOIN would exclude them.

4. Always use the alias prefix on columns. Write e.first_name and m.first_name, not just first_name. Oracle needs to know which copy of the table to read from.

5. You can use more than two copies. For a manager chain (employee -> manager -> skip-level manager), use three copies with different aliases (e, m, sm).

Section 7: JOIN with WHERE Condition

You can combine JOIN and WHERE in the same query. The JOIN connects the tables, and the WHERE clause filters the results. The WHERE condition is applied AFTER the JOIN, so you can filter on columns from either table.

```
SELECT e.first_name, e.last_name, e.salary, d.department_name FROM employees e JOIN
departments d ON e.department_id = d.department_id WHERE d.department_name = 'IT' AND
e.salary > 5000 ORDER BY e.salary DESC;
```

What happens: First, the JOIN connects EMPLOYEES with DEPARTMENTS using department_id. Then, the WHERE clause filters the results to show only employees in the IT department who earn more than 5000. The ORDER BY sorts by salary from highest to lowest. You can see how the JOIN and WHERE work together: the

JOIN brings the data together, and the WHERE selects only the rows you need.

Section 8: Joining More Than Two Tables

You can join more than two tables in one query. To do this, you add more JOIN clauses. Each JOIN connects one table to the next. The order of JOINs matters because each new JOIN can only reference tables that are already in the query.

```
SELECT e.first_name, e.last_name, d.department_name, l.city, l.state_province,
c.country_name FROM employees e JOIN departments d ON e.department_id = d.department_id
JOIN locations l ON d.location_id = l.location_id JOIN countries c ON l.country_id =
c.country_id WHERE e.department_id = 80 ORDER BY e.last_name;
```

What happens: This query joins FOUR tables together. First, EMPLOYEES joins with DEPARTMENTS on department_id. Then, DEPARTMENTS joins with LOCATIONS on location_id. Finally, LOCATIONS joins with COUNTRIES on country_id. The WHERE clause filters to show only Sales department employees (dept 80). The result shows the employee name, department name, city, state, and country - all in one result.

Think of it like a chain: EMPLOYEES -> DEPARTMENTS -> LOCATIONS -> COUNTRIES. Each link in the chain is one JOIN with one ON condition. The chain of connections goes from the employee to their department, then to the office location, and finally to the country.

Section 9: JOIN Types - Quick Comparison

This table summarizes all JOIN types. It shows which rows each JOIN keeps from the left and right tables. Use this table to quickly remember the difference between each type.

JOIN Type	Left Table	Right Table	Unmatched Rows
INNER JOIN	Only matched	Only matched	Excluded from both
LEFT JOIN	ALL rows	Only matched	Right unmatched excluded; left unmatched shown with NULL
RIGHT JOIN	Only matched	ALL rows	Left unmatched excluded; right unmatched shown with NULL
FULL OUTER JOIN	ALL rows	ALL rows	All unmatched shown with NULL
CROSS JOIN	ALL rows	ALL rows	Every row paired with every row
Self JOIN	Same table (alias 1)	Same table (alias 2)	Depends on LEFT or INNER

Section 10: Practice Queries

Try these queries on your own. Each query uses a different JOIN type. Write them in Oracle SQL Developer and check the results.

1. INNER JOIN - Employee with Department

Show first_name, last_name, and department_name for all employees who have a department.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
ORDER BY d.department_name;
```

2. LEFT JOIN - Find Employees with No Department

Show employees who do NOT have a department (department_name will be NULL).

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d
  ON e.department_id = d.department_id
WHERE d.department_name IS NULL;
```

3. RIGHT JOIN - Find Empty Departments

Show departments that have no employees.

```
SELECT d.department_name, e.first_name
FROM employees e
RIGHT JOIN departments d
  ON e.department_id = d.department_id
WHERE e.employee_id IS NULL;
```

4. Self JOIN - Employee and Manager Names

Show each employee name with their manager name.

```
SELECT e.first_name || ' reports to ' || m.first_name
  AS "Reporting Line"
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id
ORDER BY e.employee_id;
```

5. Multi-Table JOIN - Employee with Location

Show employee name, department name, city, and country.

```
SELECT e.first_name, e.last_name,
      d.department_name, l.city,
      c.country_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN locations l ON d.location_id = l.location_id
JOIN countries c ON l.country_id = c.country_id
ORDER BY e.last_name;
```

6. JOIN with WHERE - IT Employees with High Salary

Show IT department employees who earn more than 5000.

```
SELECT e.first_name, e.last_name, e.salary,
      d.department_name
```

```
FROM employees e
JOIN departments d
  ON e.department_id = d.department_id
WHERE d.department_name = 'IT'
  AND e.salary > 5000
ORDER BY e.salary DESC;
```

Important Notes

Remember these rules when you write JOIN queries. They will help you avoid common mistakes.

Rule	Explanation
Use table aliases	Always use short names for tables (like "e" for EMPLOYEES, "d" for DEPARTMENTS). This makes your query shorter and easier to read.
Specify columns with alias	When two tables have a column with the same name, always write the alias before the column name. For example: e.department_id, not just department_id.
NULL never equals NULL	A JOIN condition like e.department_id = d.department_id will never match if both values are NULL. That is why employees with NULL department_id do not appear in INNER JOIN results.
Use LEFT JOIN to keep all data	When you want ALL rows from one table, use LEFT JOIN. This ensures no data is lost. You can add WHERE ... IS NULL to find the unmatched rows.
Be careful with CROSS JOIN	CROSS JOIN multiplies the row count. If table A has 100 rows and table B has 50 rows, the result has 5,000 rows. Always check the row count before running a CROSS JOIN.
Column aliases use double quotes	In Oracle, column aliases with spaces need double quotes: AS "Full Name". Single quotes are only for string values like 'IT'.